

TP5 - Bell-LaPadula

Gabriel Mazet - L3 Informatique - 2025/2026

1. Introduction

Le but du TP est d'implémenter en Python un contrôleur d'accès qui applique les règles de Bell-LaPadula (sécurité simple et sécurité étoile) en s'appuyant sur la classe `SecurityLevel` fournie. J'ai complété le fichier `bell-laPadulla_init.py` avec deux nouvelles classes `Sujet` et `Objet`, et j'ai rempli la classe `Controleur` qui était laissée vide.

En lisant le fichier de départ je suis tombé sur une erreur dans `SecurityLevel.compare`: la ligne utilisait `contexts.compare(...)` au lieu de `self.contexts.compare(...)`, ce qui faisait planter toute comparaison avec une `NameError`. J'ai corrigé cette ligne, sinon les méthodes `read` et `write` ne peuvent pas s'exécuter. Il y avait aussi `super().__init__()` au lieu de `__init__()` dans le `Controleur` initial, mais comme je redéfinis la classe entièrement à la fin du fichier ce n'est pas un problème.

2. Structure des données

Classe Sujet

Un sujet correspond à un processus. Je lui ai donné un identifiant `sid`, un groupe `group` (comme l'`uid/gid` Unix), un niveau courant et un niveau maximal (utile pour la partie 5), un drapeau `trusted`, et deux ensembles pour suivre les objets ouverts en lecture et en écriture.

Classe Objet

Un objet a un identifiant, un propriétaire (un `sid`), un groupe, un niveau de sécurité et un drapeau `trusted` qui sert pour les programmes (partie 3).

Controleur

Le contrôleur stocke deux dictionnaires (`self.sujets` et `self.objets`) plus la matrice DAC. J'ai choisi de représenter la matrice par un dictionnaire dont les clés sont des couples (`sid, oid`) et les valeurs des ensembles de droits `'r'`, `'w'`, `'x'`. C'est plus simple à mettre à jour qu'une vraie matrice 2D. La méthode `grant` sert à ajouter des droits.

Pour rester proche de Unix j'ai décidé que le propriétaire d'un objet a tous les droits automatiquement (sans avoir à remplir la matrice). Ça simplifie `touch` qui sinon devrait ajouter trois entrées DAC à la création.

3. Partie 2 : sécurité simple et sécurité *

Pour `read`, je vérifie d'abord que le sujet a le droit DAC `r`, puis que son niveau est supérieur ou égal à celui de l'objet (No-Read-Up). Concrètement `compare` renvoie 1 ou 0 si c'est OK, -1 si c'est un read-up, et `None` si les niveaux sont incomparables (par exemple deux contextes disjoints). Dans les deux derniers cas je renvoie `False`.

Pour `write` c'est l'inverse (No-Write-Down): `compare` doit renvoyer -1 ou 0. Si l'accès est accordé, je rajoute l'oid à la liste des objets ouverts du sujet.

4. Partie 3 : execute et kill

La méthode `execute` prend le `sid` appelant, l'oid du programme à lancer, le `sid` du nouveau processus, son groupe et le niveau demandé. Elle commence par vérifier le droit `x` via DAC. Ensuite elle distingue les deux cas de l'énoncé:

- si le programme est `trusted`, le nouveau processus est créé directement au niveau demandé;
- sinon, le niveau demandé doit être inférieur ou égal au niveau actuel du sujet appelant.

La méthode `kill` partage le même droit `x`. Le cas particulier de l'énoncé ("un sujet a toujours le droit `kill` sur lui-même") est traité en premier dans la méthode. Pour `kill` sur un autre sujet je n'avais pas vraiment d'objet à associer au processus cible, donc j'ai pris une règle approchée: même groupe ou sujet `trusted`. Ce n'est pas parfait mais ça suffit pour les tests.

5. Partie 4 : objets

J'ai ajouté `close(sid, oid)` qui retire l'objet des deux ensembles `openRead` et `openWrite`.

La création se fait avec `touch(sid, oid, group=None, trusted=False)`. Le point important de l'énoncé est que *le niveau de sécurité d'un objet est celui du processus qui l'a créé*, donc je copie simplement `s.securityLevel`. Le créateur devient propriétaire.

Pour `rm`, `chmod` et `chown` je vérifie à chaque fois que l'appelant est le propriétaire. `rm` doit aussi nettoyer la matrice DAC et les ouvertures, sinon on garde des références à un objet qui n'existe plus.

6. Partie 5 : changement de niveau

La méthode `changeLevel(sid, newLevel)` n'est utilisable que par les sujets non `trusted` (l'énoncé parle bien de "sujets qui ne sont pas `trusted`"). Elle vérifie deux choses:

1. le nouveau niveau doit être inférieur ou égal au `maxLevel` du sujet (j'ai ajouté ce champ à la classe `Sujet` pour cette partie);
2. pour chaque objet déjà ouvert, le nouveau niveau doit toujours respecter la règle qui a permis l'ouverture: pour un objet ouvert en lecture il faut que `newLevel` soit toujours au-dessus, pour un objet ouvert en écriture il faut qu'il soit toujours en dessous.

Si une comparaison renvoie `None` (niveaux incomparables) je refuse aussi.

7. Tests

J'ai testé avec un treillis simple à trois niveaux:

```
public < confidentiel < secret
```

Trois objets: `doc` (public, owner alice), `rapp` (secret, owner root, alice a r/w via DAC), `shell` (programme trusted, alice a x). Deux sujets: `alice` (confidentiel, max secret, non trusted) et `root` (secret, trusted).

Sortie du script de test:

```
read doc      : True
read rapp NRU : False
write rapp    : True
write doc NWD : False
exec trusted  : True
sh1 niveau secret: secret
kill self     : True
touch f1      : True
  niveau f1   : confidentiel
chmod         : True
chown         : True
rm refusé non-owner: False
rm root      : True
changeLevel public (write rapp ouvert): True
changeLevel secret (<=max) : True
alice niveau : secret
```

Le résultat `changeLevel public ... : True` peut surprendre au premier abord puisque `alice` avait `rapp` (secret) ouvert en écriture: en fait, passer de confidentiel à public ne casse pas la règle NWD (`public <= secret` reste vrai), donc la méthode accepte correctement. Pour la bloquer il faudrait avoir un objet ouvert en lecture à un niveau supérieur au nouveau niveau.

8. Difficultés rencontrées

Le plus pénible a été la typo `contexts.compare` dans le fichier de base, j'ai mis un moment à comprendre pourquoi mes tests cassaient avant même d'arriver à mon code. J'ai aussi hésité sur le comportement de `kill` entre processus différents

puisque l'énoncé ne précise pas comment représenter le droit d'exécution sur un autre sujet, j'ai fini par retomber sur une règle proche de Unix (même groupe).

La partie 5 m'a forcé à reprendre la classe `Sujet` pour ajouter `maxLevel`: au début mes sujets n'avaient qu'un niveau constant comme dit dans l'énoncé. C'est aussi pour ça que `changeLevel` est interdit aux sujets `trusted` chez moi.